AI CAREER & INTERVIEW PREPARATION PLATFORM

Complete Django Project Blueprint • Build • Test • Deploy • Sell

Technology: Django + Python + PostgreSQL + Bootstrap 5 + AI API

Includes: Modules, models, UI pages, reports, print formats, commands, AI architecture, security, deployment and selling

Step-by-step reference for an individual developer

How to use this guide

Build in phases. First finish the MVP: accounts, student profile, resume builder, AI resume analyzer, text mock interview and interview evaluation. Then add college/recruiter features, payments and production deployment.

Product positioning

Product: AI Career & Interview Preparation Platform
Customers: Students, colleges, placement cells, training institutes, recruiters
Value: Resume improvement + AI interview practice + career readiness + placement tracking

1. User roles

• Visitor: landing page, features, pricing, register/login

• Student: profile, resume, assessment, interviews, feedback, roadmap, jobs

• College Placement Officer: organization, batches, students, drives, placement analytics

• Trainer/Mentor: courses, assessments, student review and feedback

• Recruiter: jobs, applicants, shortlist and hiring stages

• Organization Admin: branding, users, plans, quotas and reports

• Platform Super Admin: tenants, billing, AI usage, support and audit

2. Django folder architecture

```
AI_Career_Platform/
■■■ manage.py
■■■ config/                          # settings, urls, ASGI, WSGI
■■■ apps/
■   ■■■ accounts/                    # custom User, auth, onboarding
■   ■■■ organizations/                # colleges, institutes, tenants
■   ■■■ students/                     # profile, education, skills
■   ■■■ resumes/                      # builder, versions, templates
■   ■■■ ai_services/                  # AI provider, prompts, usage
■   ■■■ interviews/                   # sessions, questions, answers
■   ■■■ assessments/                  # tests, attempts, results
■   ■■■ learning/                     # courses, roadmap, progress
■   ■■■ jobs/                         # companies, jobs, applications
■   ■■■ placements/                   # drives, placement records
■   ■■■ subscriptions/                # plans, billing, quotas
■   ■■■ notifications/                # email, in-app notifications
■   ■■■ analytics/                    # reports, dashboards
■■■ templates/                        # shared + app templates
■■■ static/css, static/js, static/images
■■■ media/                            # local development only
■■■ requirements.txt
■■■ .env                              # secrets; never commit
```

Begin with flat app folders if package nesting feels confusing. Create the custom User model before the first migrate; changing it later is difficult.

3. Modules and core models

Accounts & access

• User (email login), EmailVerification, PasswordReset, LoginAudit

Organizations

• Organization, OrganizationMembership, Department, Batch, BrandingSettings

Student management

• StudentProfile, Education, Experience, Project, Certification, Skill, StudentDocument

Resume management

• Resume, ResumeVersion, ResumeTemplate, ResumeAnalysis, ResumeExport

AI services

• PromptTemplate, AIUsageLog, AIJob, AIOutputReview

Interview practice

• InterviewSession, InterviewQuestion, InterviewAnswer, InterviewEvaluation, Transcript, RecordingMetadata

Question bank

• QuestionBankItem, QuestionTag, QuestionRubric, QuestionSet

Assessments

• Assessment, AssessmentSection, AssessmentQuestion, AssessmentOption, AssessmentAttempt, AssessmentResponse, AssessmentResult

Learning

• Course, Lesson, Enrollment, LessonProgress, LearningRoadmap, RoadmapItem

Jobs & placement

• Company, JobOpening, JobSkill, JobApplication, ApplicationStageHistory, SavedJob

• PlacementDrive, DriveEligibility, PlacementRecord, OfferDocument

Subscription & platform

• Plan, PlanFeature, Subscription, Invoice, PaymentTransaction, UsageQuota, Coupon

• Notification, EmailTemplate, NotificationPreference, ActivityLog, AuditEvent, ExportJob

4. Important fields and relationships

StudentProfile

• user (OneToOne User); organization (optional); student_code; phone; photo

• college, department, degree, graduation_year, current_year, optional CGPA

• technical_skills, career_goal, preferred_roles, profile_completion, visibility_settings

Resume / ResumeVersion

• Resume: owner, organization, title, template, status, current_version

• ResumeVersion: resume, version_number, structured_content JSON, created_at, change_summary

• Sections: summary, education, experience, projects, skills, certifications, achievements

• ResumeAnalysis: version, target job description, ATS score, keyword match, missing keywords, suggestions, AI model, date

InterviewSession

• student, organization, type (technical/HR/behavioral/mixed), target_role, experience_level, difficulty

• status (created/in_progress/completed/cancelled), started_at, completed_at, duration, question_count

• overall_score, technical_score, communication_score, clarity_score, relevance_score

• feedback_summary, strengths, improvement_areas, recommended_actions, transcript consent/status

Question and Answer

• Question: prompt, category, skill, job_role, difficulty, rubric, reference answer, time_limit, points

• Answer: session, question, answer_text, optional audio, transcript, score, feedback, evaluated_at

• Keep confidential rubrics and answer keys server-side until an attempt ends

Job and application

• JobOpening: company, creator, title, description, location, workplace_type, employment_type, experience, salary range, deadline, status

• JobSkill: skill, proficiency, required/optional

• JobApplication: job, student, resume_version, cover_letter, status, applied_at, restricted recruiter_notes, stage history

PlacementRecord

• student, organization, company, job_title, annual CTC + currency, offer_date, joining_date, status, private offer_letter

• Offer documents accessible only to the student and authorized placement officer

5. Pages / custom UI routes

• Landing / : hero, product benefits, feature cards, demo CTA, pricing, FAQ

• Register /accounts/register/ : Student/College/Recruiter selection, email, password, consent

• Login /accounts/login/ : email/password, reset password

• Onboarding /onboarding/ : role-based profile setup, skills, career goal, consent

• Student Dashboard /student/dashboard/ : readiness score, KPIs, trend, recommendations, recent interviews/jobs

• Profile /student/profile/ : education, skills, projects, certifications, completion meter

• AI Career Advisor /career/advisor/ : goal, career options, skill gaps, roadmap, save advice

• Resume Builder /resumes/builder/ : section editor, templates, preview, autosave, PDF download

• Resume Analyzer /resumes/analyzer/ : resume + job description; ATS score, matched/missing keywords

• Mock Interview Setup /interviews/setup/ : role, type, level, difficulty, duration, text/voice, consent

• Live Interview /interviews/session// : question, timer, answer, controls, progress, finish

• Interview Result /interviews/result// : scores, question feedback, strengths, improvement, next steps

• Interview History /interviews/history/ : filters, trend, reports, retake

• Assessments /assessments/ and /assessments/result// : tests, attempts, scores, explanations

• Learning Roadmap /learning/roadmap/ : milestones, progress, resources, mark complete

• Courses /learning/courses/ : catalog, details, enroll, lessons, completion certificate

• Job Portal /jobs/ : search, filters, save, eligibility, apply

• Job Details /jobs// : description, skills, employer, deadline, apply

• My Applications /jobs/my-applications/ : stage tracker, status history

• College Dashboard /college/dashboard/ : student count, readiness, placement %, department analytics

• College Students /college/students/ : CSV import, batch/department filters, readiness

• Placement Drives /college/drives/ : eligibility, company/job, applicants, shortlist

• Recruiter Dashboard /recruiter/dashboard/ : jobs, applicant pipeline, stages

• Recruiter Applicants /recruiter/jobs//applicants/ : candidates, permitted resume, filters, stage update

• Organization Settings /organization/settings/ : logo, colors, members, roles, branding

• Pricing /pricing/ and /billing/ : plans, quotas, payments, invoices, renewal

• Platform Admin /platform-admin/ : tenants, plans, AI spend, support, audit

6. Reports and dashboard charts

• Career Readiness: student, college, department, batch, profile %, skills, resume score, interview average, readiness band

• Interview Performance: student, date, role, type, overall/technical/communication score, duration, attempts

• Question-wise Interview Analysis: session, question, skill, rubric, score, feedback

• Interview Score Trend: student, attempt count, date, score trend; date/type filters

• Skill Gap Analysis: student/department, target role, required/current skill, gap level

• Resume ATS Analysis: student, resume version, target role, score, keyword match, date

• Assessment Results: assessment, student, score, %, duration, attempt, pass/fail

• Job Application Pipeline: job/company, counts by stage, shortlisted/interviewed/offered/hired

• Job-wise Applicants: applicant, education, skill match, resume, date, status

• Placement Summary: batch/department, eligible, placed, placement %, average/highest CTC

• Company-wise Placement: company, role, selections, CTC band, offer date

• Course Completion: course, enrolled, completed, completion %, assessment average

• Subscription & AI Usage: tenant, plan, seats, requests, tokens/cost, quota, renewal

Charts: Interview score trend (line), readiness bands (bar), skill gaps (horizontal bar), application funnel, placement rate by batch, AI usage/cost by tenant. Never expose identifiable student data on public dashboards.

7. Print formats / downloadable PDFs

• ATS-friendly Resume — A4 portrait; clean single-column option; contact, summary, skills, experience, education, projects, certificates

• AI Interview Evaluation — A4 portrait; student/session, question-answer summary, rubric scores, strengths, improvements, practice recommendations, disclaimer

• Skill Assessment Result — A4 portrait; test/date, score, skill breakdown, valid percentile only if available, next steps

• Career Readiness Report — A4 portrait; profile, skill matrix, resume readiness, interview trend, roadmap

• Course Completion Certificate — A4 portrait; student, course, dates, issuer, certificate ID, verification QR/URL

• College Placement Summary — A4 landscape; batch/department totals, eligible/placed, %, companies, salary summary

• Placement Offer Summary — A4 portrait; student, company, role, CTC, date; confidential; not employer's official offer

• Job Application Summary — A4 portrait; job, applicant, resume version, applied date, current stage; omit recruiter notes

• Invoice / Receipt — A4 portrait; seller, invoice, plan/period, tax, transaction reference, payment status

PDF options: HTML-to-PDF with WeasyPrint (Windows dependencies may be needed), or printable HTML + browser Print. Store generated private files in protected storage, not public static.

8. UI design / wireframe brief (no images embedded)

• Desktop: sidebar 240–260px, topbar search/notifications/profile, content width up to ~1440px

• Mobile: collapsible drawer, responsive cards/tables, sticky primary action for interview/resume

• Visual system: neutral light background, navy/indigo primary, one accent, 8px spacing scale, 12–16px card radius

• Components: KPI cards, progress, badges, filterable tables, tabs, empty/loading/error states, toast, modal

• Accessibility: keyboard navigation, focus, contrast, labels, screen reader and reduced motion

Screen layouts

• Student dashboard: greeting + readiness hero; 4 KPIs; score trend; roadmap; recent interviews and applications

• Resume builder: left section nav; center editor; right live A4 preview; top save/template/export

• Mock interview: role/timer/progress; central AI question; answer area; voice controls; session rules/mic state

• Interview report: score hero; category cards; question accordion; strengths/improvements; roadmap CTA; print/download

• College dashboard: KPIs; placement funnel; readiness distribution; department comparison; filtered student table

• Recruiter pipeline: Kanban Applied/Screen/Interview/Offer/Hired/Rejected; candidate drawer and stage history

9. PowerShell commands — continue from your current project

You ran django-admin startproject AI_Career_Platform from C:\Shravan Projects\Django Projects. Run:

```
cd "C:\Shravan Projects\Django Projects\AI_Career_Platform"
dir

# Create virtual environment
py -m venv .venv
.\.venv\Scripts\Activate.ps1

# If activation is blocked for this terminal:
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1

# Install essentials
python -m pip install --upgrade pip
pip install django pillow python-dotenv

# Verify + test server
python -m django --version
python manage.py runserver
# Browser: http://127.0.0.1:8000/
# Stop server: Ctrl + C
```

Create apps gradually — Phase 1 MVP

```
python manage.py startapp accounts
python manage.py startapp students
python manage.py startapp resumes
python manage.py startapp ai_services
python manage.py startapp interviews
```

Phase 2 apps after MVP

```
python manage.py startapp organizations
python manage.py startapp assessments
python manage.py startapp learning
python manage.py startapp jobs
python manage.py startapp placements
python manage.py startapp subscriptions
python manage.py startapp notifications
python manage.py startapp analytics
```

If manage.py is not shown by dir, check for a nested AI_Career_Platform folder and cd there. Run commands from the directory containing manage.py. Do not run startproject again.

10. Custom User model and settings

Implement accounts.User before your first migrate, then set AUTH_USER_MODEL. Email-based login is recommended. Use Django password hashing; never store plaintext passwords.

```
# config/settings.py (illustrative)
INSTALLED_APPS = [
    # Django defaults ...
    "apps.accounts",
    "apps.students",
    "apps.resumes",
    "apps.ai_services",
    "apps.interviews",
    # expansion apps later
]
AUTH_USER_MODEL = "accounts.User"

# .env (local only; do not commit)
DJANGO_SECRET_KEY=use-a-long-random-secret
DJANGO_DEBUG=True
DJANGO_ALLOWED_HOSTS=127.0.0.1,localhost
OPENAI_API_KEY=your-provider-key
```

Use python-dotenv to load .env. Keep SECRET_KEY, AI keys, database passwords and email credentials outside source control. SQLite is fine locally; use PostgreSQL in production.

11. Migrations, admin and server

```
# after apps + models + settings
python manage.py makemigrations
python manage.py migrate

# admin account
python manage.py createsuperuser

# server
python manage.py runserver

# admin URL after adding it in config/urls.py
http://127.0.0.1:8000/admin/
```

12. URL routing plan

```
# config/urls.py conceptual
path("", views.landing, name="landing")
path("accounts/", include("apps.accounts.urls"))
path("student/", include("apps.students.urls"))
path("resumes/", include("apps.resumes.urls"))
path("career/", include("apps.ai_services.urls"))
path("interviews/", include("apps.interviews.urls"))
path("assessments/", include("apps.assessments.urls"))
path("learning/", include("apps.learning.urls"))
path("jobs/", include("apps.jobs.urls"))
path("college/", include("apps.organizations.urls"))
path("placements/", include("apps.placements.urls"))
path("billing/", include("apps.subscriptions.urls"))
```

Create urls.py in each app, use namespaces, and protect student/college/recruiter views with login + role + organization membership checks.

13. Git and .gitignore

```
git init
git add .
git commit -m "Initial Django project setup"

# .gitignore
.venv/
__pycache__/
*.py[cod]
.env
db.sqlite3
media/
staticfiles/
```

Never commit API keys, secret key, credentials, student data or private uploads.

14. AI integration architecture

• AI calls happen on Django backend; never expose provider keys in browser JS

• Service functions: analyze_resume(), generate_questions(), evaluate_answer(), recommend_career()

• Validate structured JSON; handle timeouts, retries, rate limits and provider errors

• AIUsageLog: owner, tenant, provider/model, prompt version, tokens, estimated cost, latency, timestamp

• Use Celery + Redis or managed queue for long jobs; UI states pending/processing/completed/failed

• Treat resumes/JDs as untrusted; defend against prompt injection

• Label scores as practice guidance, not objective hiring decisions; allow feedback/reporting

• Build text interviews first; voice later with explicit consent and retention/deletion controls

```
Student submits answer
 -> verify ownership + active session
 -> load question + private rubric server-side
 -> call AI service
 -> validate response schema
 -> save answer score and feedback
 -> update session aggregates
 -> return only permitted result
```

15. Security, privacy and permissions

• Django auth, CSRF, secure cookies, HTTPS, email verification, password reset

• Object-level ownership on every resume/session/file/application/report endpoint; never trust URL IDs

• Multi-tenant queries scoped to organization; add cross-tenant isolation tests

• Private files outside public web root; authenticated or signed time-limited links

• Validate file type/size, scan uploads, safe filenames; never execute uploads

• Voice consent and retention/deletion policy for audio and transcripts

• Minimize personal data; restrict recruiter access; consent and data export/delete

• Rate-limit login, reset, AI, upload and export; audit admin/recruiter actions

• Production: DEBUG=False, exact ALLOWED_HOSTS, strong secrets, HTTPS, backups, monitoring

• Review applicable Indian privacy obligations (DPDP Act/rules etc.) with qualified counsel before launch

16. Testing checklist

• Accounts: register, duplicate email, verify, login/logout, reset, inactive user

• Authorization: student cannot access another student; recruiter only permitted data; tenant isolation

• Resume: CRUD, autosave, versions, preview/PDF, invalid upload blocked

• AI: timeout, bad JSON, quota, retry, cost log, prompt injection text

• Interview: state transitions, progression, timer, save answer, finish, no duplicate evaluation

• Assessment: randomization, scoring, attempts, answer key hidden until submit

• Jobs: deadline, eligibility, duplicate application, withdraw, stage history

• Payments: success/failure/cancel, signed webhook, idempotency, refund/renewal; never trust browser success

• Reports/UI: filters, date boundaries, permissions, CSV/PDF, mobile, accessibility, error/empty/loading

17. Production deployment checklist

Recommended: Linux VPS or managed host, managed PostgreSQL, object storage for uploads, background worker/Redis if needed, email provider, monitoring and automated backups.

```
pip install gunicorn psycopg[binary] whitenoise

python manage.py check
python manage.py check --deploy
python manage.py collectstatic
python manage.py migrate

# Example; replace config if your project package differs
gunicorn config.wsgi:application --bind 0.0.0.0:8000
```

• Domain + DNS + HTTPS; Nginx/reverse proxy; Gunicorn + systemd/container

• PostgreSQL automated backups and restore test

• Static via WhiteNoise/CDN; private media via object storage

• Worker + Redis for async AI; monitor errors, spend, latency, DB and uptime

• Separate dev/staging/production; staging uses anonymized data

• CI/CD: lint, tests, security scan, deploy, migrate, smoke test, rollback plan

18. Selling the product / SaaS packaging

• Brand: product name, logo, domain, tagline, demo account

• Plans & quotas: monthly/yearly, seats, interview quota, resume analysis, storage, support

• Trial/onboarding: guided setup, sample data, demo interview, checklist

• College setup: CSV import, batches, branding, placement officer training

• Legal: terms, privacy policy, AI disclaimer, refund/cancel, data-processing terms

• Billing: gateway, invoice/receipt, signed webhook, renewals, failed payment, cancellation

• Support: support email/ticket, knowledge base, uptime/maintenance policy

• Sales: landing page, 2-minute demo, brochure, pricing comparison, case study

• Trust: privacy summary, export/delete, permissions, backup and incident process

• Customer success: usage/outcomes report, onboarding call, renewal reminders

Suggested first market

Start with one segment: small/medium training institutes or college placement cells. Pilot with 3–5 organizations and validate workflow and willingness to pay. Make AI quotas transparent because usage creates variable costs.

19. MVP acceptance checklist

• Student signup/login + email verification + profile

• Resume builder with 2–3 ATS-friendly templates + PDF export

• Resume analyzer (resume + JD) with score explanation + keyword gaps

• Text mock interview for Python/Django, HR and behavioral roles

• Interview evaluation: scores, strengths, improvements, practice plan

• Interview history + student dashboard

• Admin question bank + basic user management

• Consent, quota, AI usage/cost logging, ownership checks, tests

• Demo organization, sample data and responsive UI

20. Recommended development sequence

1. Project folder, virtual environment and packages

2. Settings, .env, .gitignore, base template, Bootstrap

3. Custom User, roles, migration and admin

4. Register/login/reset/onboarding

5. StudentProfile, profile UI and dashboard shell

6. Resume models, builder, versioning, preview, PDF

7. AI resume analyzer, usage log, quota, results

8. Question bank, interview setup and session state

9. AI answer evaluation, result page and PDF

10. Reports, filters, charts, CSV/PDF

11. Assessments, roadmap, job portal

12. Organizations, college placement and recruiter

13. Plans, payment gateway, quotas, billing

14. Security, tests, staging, production, pilot customers

21. Commands to run now

Run in PowerShell from the folder where manage.py exists:

```
cd "C:\Shravan Projects\Django Projects\AI_Career_Platform"
dir
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install django pillow python-dotenv
python manage.py runserver
```

If dir does not show manage.py, locate the inner project folder and cd there. Then run the server. Do not recreate the project.

22. Command cheat sheet

```
python manage.py runserver
python manage.py startapp accounts
python manage.py startapp students
python manage.py startapp resumes
python manage.py startapp ai_services
python manage.py startapp interviews

python manage.py makemigrations
python manage.py makemigrations accounts
python manage.py migrate
python manage.py showmigrations

python manage.py createsuperuser
python manage.py shell
python manage.py check
python manage.py check --deploy
python manage.py test
python manage.py test apps.accounts
python manage.py collectstatic

pip freeze > requirements.txt
pip install -r requirements.txt

git status
git add .
git commit -m "Implement feature"
git push
```

Final advice

Build one working feature at a time: models → admin → migrations → views/forms/templates → permissions → tests → Git commit. Do not start with voice AI or complex billing. Make text interview and resume flow stable first. This PDF is a blueprint, not production-ready code; implement and test models, AI, payments, privacy and security before real customers or student data.